

IPC Through Pipes and FIFOs (Named Pipes)

Table of Contents

1. [Introduction](#)
 2. [Background: File Descriptors](#)
 3. [Anonymous \(Ordinary\) Pipes](#)
 4. [Working Example: Parent to Child](#)
 5. [Two-Way Communication with Two Pipes](#)
 6. [Pipes in the Shell](#)
 7. [Named Pipes \(FIFOs\)](#)
 8. [Working Example: FIFO Writer and Reader](#)
 9. [Pipe vs. FIFO Comparison](#)
 10. [Important Behaviours to Know](#)
 11. [Advantages and Disadvantages](#)
 12. [Real-World Use Cases](#)
 13. [Summary](#)
 14. [References](#)
-

1. Introduction

A **pipe** is one of the oldest and simplest IPC mechanisms in Unix and Linux. It is a **one-way channel** in the kernel: one process writes data into one end, and another process reads it from the other end.

Analogy: A pipe is literally like a water pipe. Data goes in at one end and comes out the other in the same order.

```
Writer process                                Reader process
+-----+                                     +-----+
| write() ---+--> | PIPE (kernel buffer) | ---+--> read() |
+-----+                                     +-----+
| first in, first out |                                     |
+-----+
```

There are two kinds:

- **Anonymous pipes:** no name, only for related processes (such as parent and child).

- **Named pipes (FIFOs):** have a name in the file system, so *any* processes can use them.

2. Background: File Descriptors

Before using pipes, you need to know **file descriptors (FDs)**. An FD is a small integer the OS gives your process to refer to an open file or channel.

FD	Meaning
0	Standard input (<code>stdin</code>)
1	Standard output (<code>stdout</code>)
2	Standard error (<code>stderr</code>)
3+	Files, pipes, sockets you open

Pipes are used through the same `read()` and `write()` calls as files. That is why a pipe is described as a "file-like" IPC method.

3. Anonymous (Ordinary) Pipes

3.1 The `pipe()` System Call

```
#include <unistd.h>

int pipe(int fd[2]);
```

`pipe()` creates a pipe and fills the array `fd` with two file descriptors:

Element	Meaning
<code>fd[0]</code>	The read end
<code>fd[1]</code>	The write end

It returns `0` on success and `-1` on error.

Memory trick: `fd[0]` is like the number **0** in "Output of the pipe" (where you read from), and `fd[1]` is where you put data **in**.

3.2 How It Works with `fork()`

A pipe by itself is useless within one process. The trick is to combine it with `fork()` :

1. The parent calls `pipe()` , getting `fd[0]` and `fd[1]` .
2. The parent calls `fork()` . The child gets a **copy** of both file descriptors, pointing to the **same pipe**.
3. Now both processes have both ends. Each process **closes the end it does not use**.
4. One process writes, the other reads.

Step 1: after `pipe()`

```
Parent
fd[0] fd[1]
  \   /
  [ PIPE ]
```

Step 2: after `fork()`

```
Parent          Child
fd[0] fd[1]    fd[0] fd[1]
  \   \        /   /
  +---[ PIPE ]-----+-----+
```

Step 3: parent writes, child reads (unused ends closed)

```
Parent (writer)          Child (reader)
close(fd[0])             close(fd[1])
write(fd[1], ...) -----> [ PIPE ] -----> read(fd[0], ...)
```

3.3 Why Close the Unused Ends?

This is very important:

- If the reader keeps the **write end** open, `read()` will **never see end-of-file** and may block forever.
- If a writer writes when **no reader** has the read end open, the OS sends the `SIGPIPE` signal and the process dies.

Rule: every process closes the pipe end it does not use.

4. Working Example: Parent to Child

The parent sends a message to the child through a pipe.

File: `code/pipes/pipe_demo.c`

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    char buffer[100];

    // 1. Create the pipe
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    // 2. Create the child process
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // ----- CHILD: reader -----
        close(fd[1]); // close unused write end

        ssize_t n = read(fd[0], buffer, sizeof(buffer) - 1);
        if (n > 0) {
            buffer[n] = '\0'; // make it a valid C string
            printf("[Child ] Received: %s\n", buffer);
        }

        close(fd[0]);
        exit(0);
    } else {
        // ----- PARENT: writer -----
        close(fd[0]); // close unused read end

        const char *msg = "Hello child, this is your parent!";
        printf("[Parent] Sending: %s\n", msg);
        write(fd[1], msg, strlen(msg));

        close(fd[1]); // signals end-of-data to
        reader
        wait(NULL); // wait for child to finish
    }
}

```

```
    return 0;
}
```

Compile and run:

```
gcc pipe_demo.c -o pipe_demo
./pipe_demo
```

Sample output:

```
[Parent] Sending: Hello child, this is your parent!
[Child ] Received: Hello child, this is your parent!
```

Line-by-line highlights:

Code	Explanation
<code>pipe(fd)</code>	Creates the pipe. Must be done before <code>fork()</code> .
<code>close(fd[1])</code> in child	The child only reads, so it closes the write end.
<code>buffer[n] = '\0'</code>	<code>read()</code> does not add a null terminator. We add it ourselves.
<code>close(fd[1])</code> in parent	Once all write ends are closed, the reader gets end-of-file.
<code>wait(NULL)</code>	The parent waits so the child is not left as a zombie.

5. Two-Way Communication with Two Pipes

A pipe is **half-duplex**: data flows in **one direction only**. For two-way communication, use **two pipes**.

```
                pipe1 (parent -> child)
Parent  =====> Child
        <=====
                pipe2 (child -> parent)
```

```
int p2c[2]; // parent to child
int c2p[2]; // child to parent
pipe(p2c);
pipe(c2p);
```

```
// after fork():
// Parent: close(p2c[0]); close(c2p[1]); // writes p2c[1], reads c2p[0]
// Child : close(p2c[1]); close(c2p[0]); // reads p2c[0], writes c2p[1]
```

6. Pipes in the Shell

You already use pipes every time you write `|` in a terminal.

```
ls -l | grep ".txt" | wc -l
```

```
+-----+ stdout +-----+ stdout +-----+
|  ls  | -----> | grep | -----> |  wc  | ----> terminal
+-----+ pipe 1  +-----+ pipe 2  +-----+
```

- The shell creates a pipe between each pair of commands.
- The output (`stdout`) of the left command becomes the input (`stdin`) of the right command.
- This is done with the `dup2()` system call, which redirects a file descriptor. For example, `dup2(fd[1], STDOUT_FILENO)` makes the process's standard output go into the pipe.

7. Named Pipes (FIFOs)

7.1 The Problem with Anonymous Pipes

Anonymous pipes need a common ancestor (via `fork()`). They cannot connect two **unrelated** programs started separately. They also disappear when the processes end.

7.2 The Solution: FIFO

A **FIFO** (First In, First Out), or **named pipe**, is a special file in the file system that acts as a pipe. Any process that knows its name can open it.

```
Process A (unrelated)                                Process B (unrelated)
+-----+
| write() -----> /tmp/myfifo [ FIFO special file ] -+--> read() |
+-----+ (data stays in kernel) +-----+
```

7.3 Creating a FIFO

From the shell:

```
mkfifo /tmp/myfifo
ls -l /tmp/myfifo      # the file type shows 'p' for pipe
```

From C:

```
#include <sys/types.h>
#include <sys/stat.h>

int mkfifo(const char *pathname, mode_t mode);
```

- `pathname` : where the FIFO file will be created.
- `mode` : permissions, such as `0666` .
- Returns `0` on success, `-1` on error.

7.4 Using a FIFO

Once created, a FIFO is used like a file: `open()` , `read()` / `write()` , `close()` . To remove it, use `unlink()` in C or `rm` in the shell.

Note: Opening a FIFO for reading **blocks** until another process opens it for writing (and vice versa). This is normal and acts as a natural synchronization.

8. Working Example: FIFO Writer and Reader

Two independent programs communicate through `/tmp/myfifo` .

8.1 Writer

File: `code/pipes/fifo_writer.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_PATH "/tmp/myfifo"
```

```

int main(void) {
    // Create the FIFO (ignore the error if it already exists)
    mkfifo(FIFO_PATH, 0666);

    printf("[Writer] Waiting for a reader to connect...\n");
    int fd = open(FIFO_PATH, O_WRONLY);          // blocks until a reader opens it
    if (fd == -1) {
        perror("open");
        exit(1);
    }

    const char *msg = "Hello through the FIFO!";
    write(fd, msg, strlen(msg) + 1);             // +1 to include the '\0'
    printf("[Writer] Sent: %s\n", msg);

    close(fd);
    return 0;
}

```

8.2 Reader

File: code/pipes/fifo_reader.c

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO_PATH "/tmp/myfifo"

int main(void) {
    char buffer[100];

    int fd = open(FIFO_PATH, O_RDONLY);          // blocks until a writer opens it
    if (fd == -1) {
        perror("open");
        exit(1);
    }

    ssize_t n = read(fd, buffer, sizeof(buffer));
    if (n > 0) {
        printf("[Reader] Received: %s\n", buffer);
    }

    close(fd);
}

```

```
    unlink(FIFO_PATH);                // remove the FIFO file
    return 0;
}
```

8.3 Run

Open **two terminals**:

```
# Terminal 1
gcc fifo_writer.c -o fifo_writer
./fifo_writer

# Terminal 2
gcc fifo_reader.c -o fifo_reader
./fifo_reader
```

Sample output:

Terminal 1:

```
[Writer] Waiting for a reader to connect...
[Writer] Sent: Hello through the FIFO!
```

Terminal 2:

```
[Reader] Received: Hello through the FIFO!
```

Try this without any C code:

```
mkfifo /tmp/testfifo
# Terminal 1:
cat /tmp/testfifo
# Terminal 2:
echo "Hello FIFO" > /tmp/testfifo
```

9. Pipe vs. FIFO Comparison

Feature	Anonymous Pipe	Named Pipe (FIFO)
Has a name in the file system	No	Yes
Created with	<code>pipe()</code>	<code>mkfifo()</code>
Communication between	Related processes only (parent, child, siblings)	Any processes
Lifetime	Until all descriptors are closed	Name stays until deleted; data lives in kernel only while open
Direction	One-way	One-way (use two FIFOs for two-way)
Setup	Must be created before <code>fork()</code>	Any process can open by name
Typical use	Shell pipelines, parent-child communication	Independent programs, client-server on same machine

10. Important Behaviours to Know

Situation	What happens
Read from an empty pipe while a writer exists	<code>read()</code> blocks (waits)
Read from an empty pipe with all write ends closed	<code>read()</code> returns <code>0</code> (end-of-file)
Write to a full pipe	<code>write()</code> blocks until space is available
Write when no reader exists	<code>SIGPIPE</code> signal sent, and <code>write()</code> fails with <code>EPIPE</code>
Pipe capacity	Typically 64 KB on modern Linux (65536 bytes)
Writes up to <code>PIPE_BUF</code> (4096 bytes on Linux)	Atomic : not mixed with other writers' data
Data order	Always first in, first out . You cannot skip ahead.

11. Advantages and Disadvantages

Advantages

- **Very simple** to create and use
- Uses ordinary `read()` and `write()` calls
- Kernel handles **synchronization** (blocking when empty or full)
- Data is delivered **in order**
- FIFOs allow communication between **unrelated** processes
- Foundation of shell command pipelines

Disadvantages

- **One-way only** (need two pipes for two-way communication)
- Anonymous pipes work only between **related** processes
- Works only on the **same machine**
- Data is a plain **byte stream** with no message boundaries, so you must design your own format
- **Limited capacity**
- No random access: data can only be read in order, and once it is read it is gone
- Not suitable for broadcasting to several readers, because each byte is read by only one reader

12. Real-World Use Cases

Use Case	Example
Shell pipelines	<code>cat log.txt \ grep error \ sort \ uniq</code>
Parent-child communication	A program creating a worker process and collecting its output
Logging	Applications write to a FIFO and a logger process reads from it
Running external programs from code	<code>popen()</code> in C and <code>subprocess</code> in Python use pipes internally
Simple client-server on one machine	Small daemons that accept commands through a FIFO

13. Summary

- A **pipe** is a one-way, first-in-first-out byte channel in the kernel.
 - **Anonymous pipes** (`pipe()`) work between related processes and are created before `fork()` . Always **close the unused ends**.
 - **Named pipes / FIFOs** (`mkfifo()`) live in the file system and connect **unrelated** processes.
 - Pipes are **half-duplex**, so use two pipes for two-way communication.
 - The shell's `|` operator is built on pipes.
-

14. References

1. Silberschatz, Galvin, Gagne. *Operating System Concepts* (Chapter 3: Pipes).
2. Linux manual pages: `man 2 pipe` , `man 3 mkfifo` , `man 7 pipe` , `man 7 fifo` .
3. Kerrisk, M. *The Linux Programming Interface* (Chapter 44: Pipes and FIFOs).
4. GeeksforGeeks: Pipe and Named Pipe (FIFO) in C.